

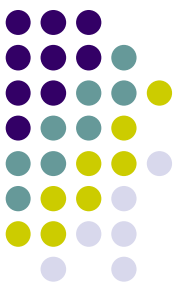
Factory Pattern

Architecture of Computer Games
SE 456

Ed Keenan

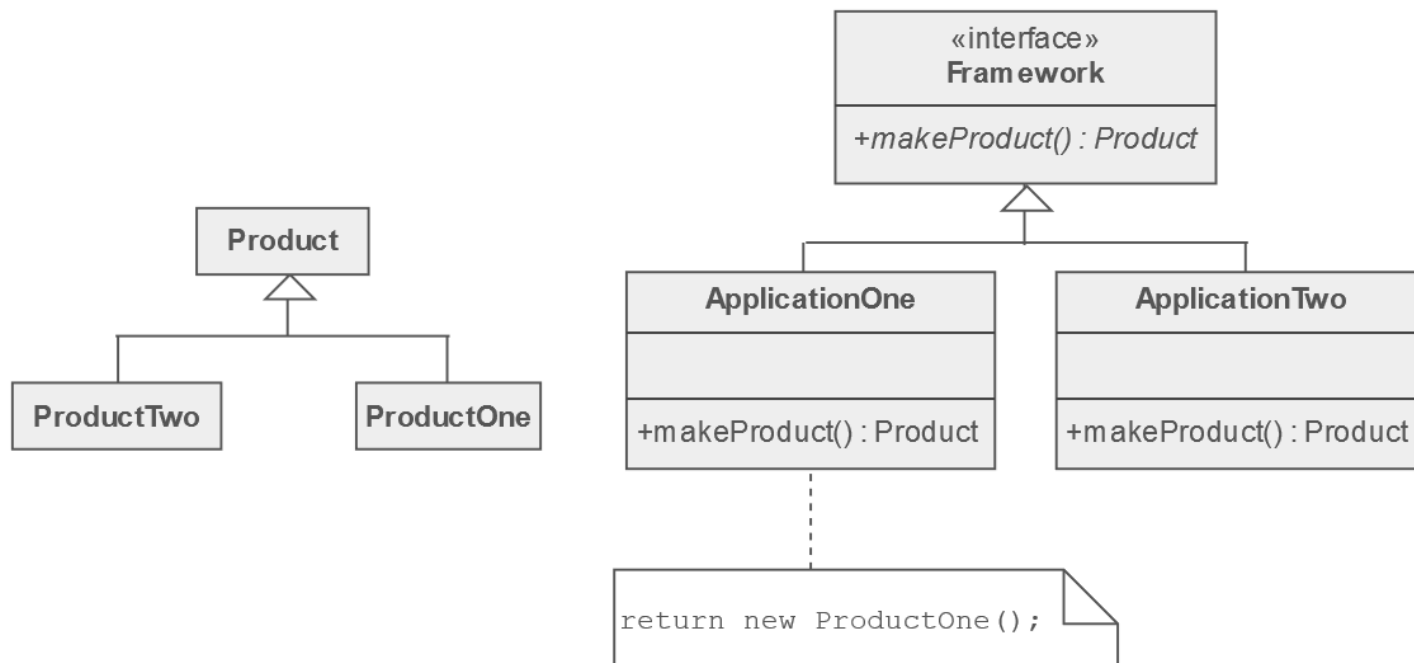
13 January 2021

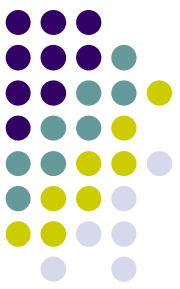
13.0.8.3.5 Mayan Long Count



Factory

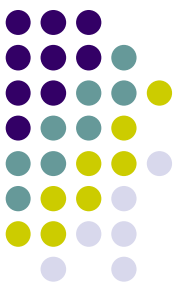
- Intent
 - Creates objects without exposing instantiation logic to the client.
 - Refers to the newly created object through a common interface





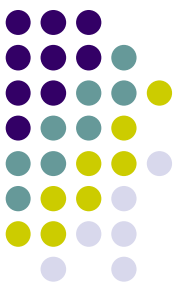
Factories – Novice approach

- New developers often lump object creation with object instantiation.
 - Often
 - Instantiate object before you know what you want
 - How you intend on using the object
 - Causing problems



Factories - Creational

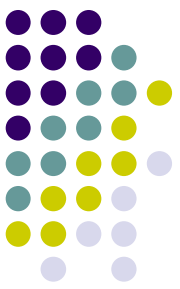
- A factory is a method, object or anything else to **instantiate** an object
- Factory is a *Creational Pattern*
 - Used to create objects
- Factories **hides** the variation
 - Let the variation be done through factories instead by explicit means



Factories – Philosophy

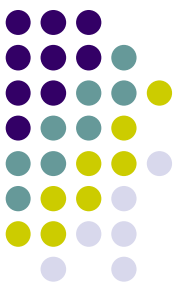
- Consider what you need to have in your system before you concern yourself with how to create it...
- We define our factories after decide what our objects are and do.

First we decide what our objects are and how they work, then decide how to instantiate them.

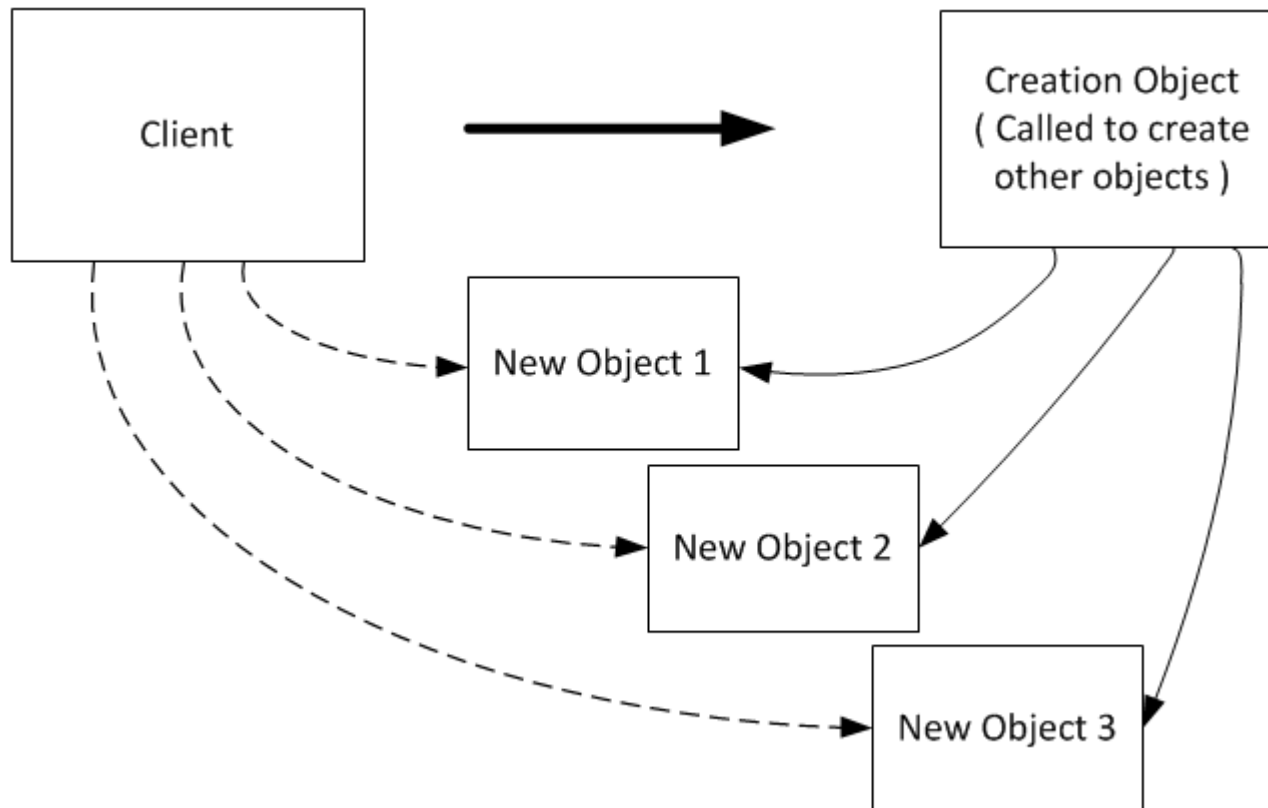


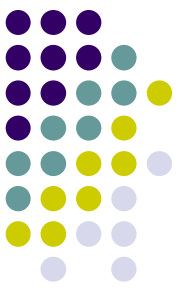
Factories - Responsibilities

- Splitting development into two steps
 1. Define the Objects and how they work together
 2. Write factories that instantiate the correct objects for the right situation and manage existing object if they are shared.
- Factories
 - Increase Cohesion
 - Loosen coupling
 - Aid in testing



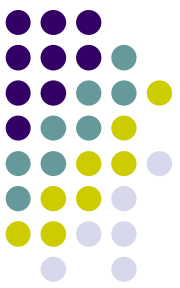
Factories - Responsibilities





Factories - Responsibilities

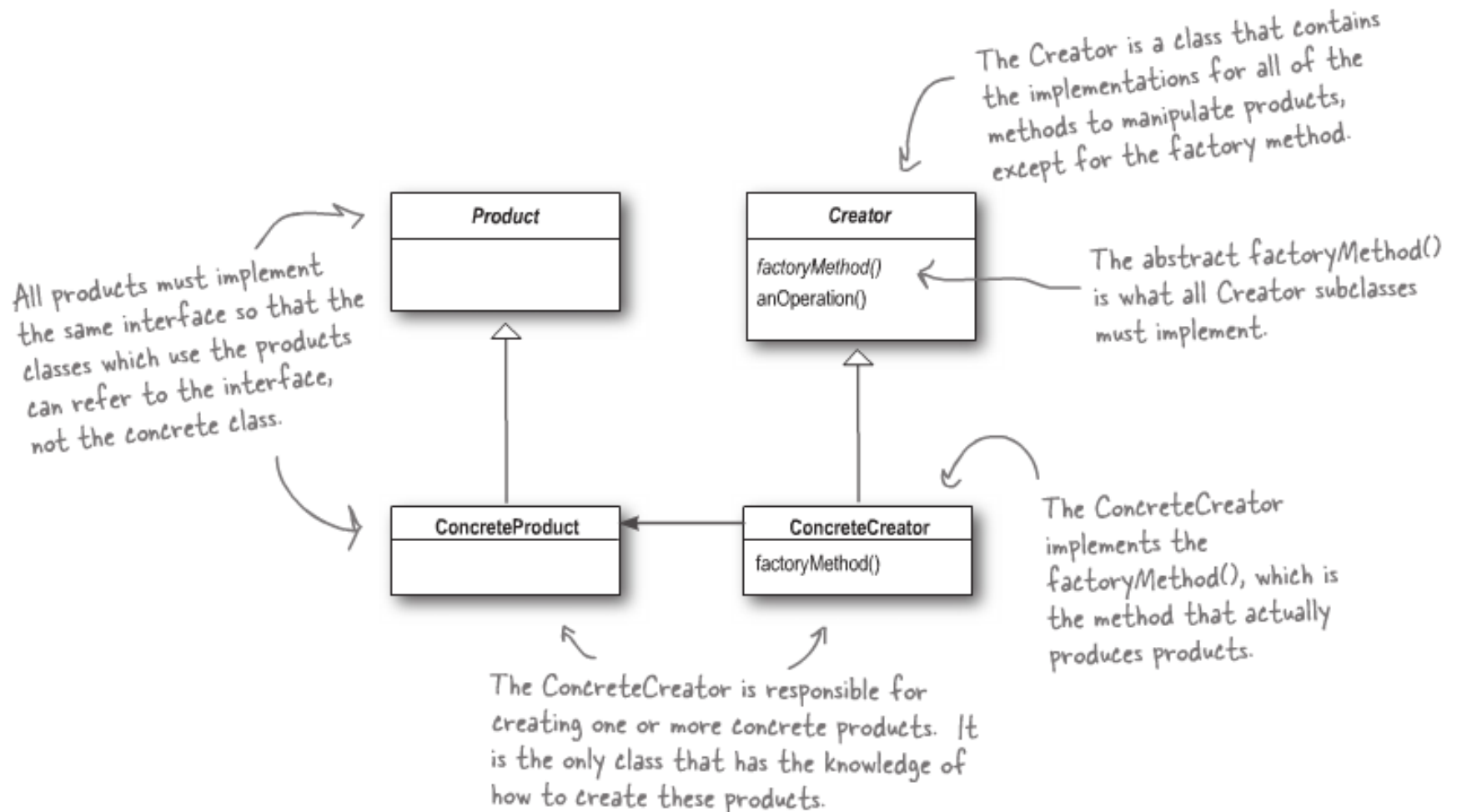
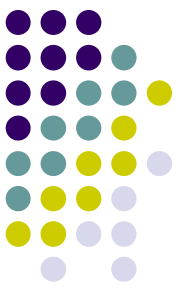
- General Object Oriented Rule
 - Object should either:
 - Make and/or Manage other objects
 - OR
 - Use other objects
 - Never do both.
- You end up with less coupling because there is a clear division of labor.

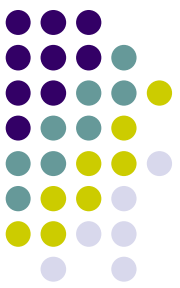


Factory Method

- Factory method
 - Creating objects (products) without specifying the exact class of object that will be created
 - Factory method design pattern handles this problem by defining a **separate method** for creating the objects
 - Subclasses can then override to specify the derived type of product that will be created
- Implementation
 - Defines an interface for creating an object
 - Lets subclasses decide which class to instantiate
 - Factory Method lets a class **defer** instantiation to subclasses.

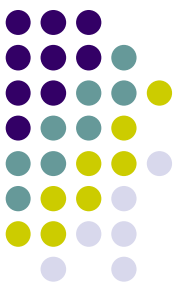
Factory





Factories in Games

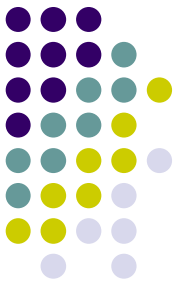
- Creating objects in a fire and forget way?
 - Should be used everywhere
 - Need to create objects
 - Might be complex to create
 - Repetitive creation
 - Creating loose objects
 - Hiding behind a resource pool
 - Examples
 - Sprites (2D objects)
 - Sounds
 - Combo



Factory

- HFDP Chapter 4 – Factory Pattern
 - in Head First Design Patterns book
 - Campy – but get's the message across
- Reference Design Patterns
 - DoFactory – Design Patterns in C#
 - Try those demos
- Code Demo

Thank You!



- Questions?

